

SDK Integration Guide: MagicData Python Bridge

1. Overview

The **MagicData Python Bridge** is a high-performance C-based library designed to embed a Python interpreter directly into native C/C++ applications. It provides a fully **isolated environment**, meaning it does not require a global Python installation on the host machine.

2. SDK Structure

The SDK package is organized as follows:

- **include/**: Contains the C header file (md_python_engine.h) and the static import library (MD_BridgeLib.lib).
- **bin/**: Contains the Runtime Environment, including:
 - MD_BridgeLib.dll: The core bridge logic.
 - python311.dll & python311._pth: The embedded Python engine.
 - Lib/ & DLLs/: The standard library and third-party packages (e.g., Pandas, Requests).

3. Integration Steps (Visual Studio)

A. Project Configuration

To link the SDK with your C/C++ project, follow these steps:

1. **Header Search Path**: Go to Project Properties -> C/C++ -> General -> Additional Include Directories and add the path to the SDK include folder.
2. **Library Search Path**: Go to Linker -> General -> Additional Library Directories and add the path to the SDK include folder (where the .lib file is located).
3. **Linker Dependencies**: Go to Linker -> Input -> Additional Dependencies and add: MD_BridgeLib.lib.

B. Deployment Automation (Post-Build Event)

To ensure all necessary runtime files are present in your output directory, add the following command to Project Properties -> Build Events -> Post-Build Event:

```
xcopy /y /e /s "PATH_TO_SDK\bin\*" "$(TargetDir)"
```

Note: Replace PATH_TO_SDK with the actual path on your machine.

4. API Reference

The SDK exposes a simple, thread-safe function to execute Python scripts.

Function Signature

```
void RunPythonScript(const char* scriptPath);
```

Note on scriptPath:

The function accepts both relative and absolute paths:

- **Relative Path:** e.g., "script.py" (looks for the file in the same directory as the executable).
- **Absolute Path:** e.g., "C:\\Scripts\\my_logic.py".
- *Important:* When using absolute paths in C/C++, remember to use double backslashes (\\) to escape the path correctly.

Example Usage

```
#include "md_python_engine.h"
```

```
int main() {  
    // Executes a script located in the application directory  
  
    // Example 1: Using a relative path (Recommended for portability)  
    RunPythonScript("logic_test.py");  
  
    // Example 2: Using a full absolute path  
    RunPythonScript("C:\\MagicData\\Scripts\\data_processor.py");  
  
    return 0;  
}
```

5. Technical Highlights

- **Isolated Runtime:** The bridge uses a redirected path logic via python311._pth, ignoring local environment variables to prevent version conflicts.
- **Third-Party Support:** Pre-configured with site-packages support. You can add new libraries by installing them directly into bin/Lib/site-packages.
- **Error Handling:** Built-in Python exception trapping. If a module is missing or a script fails, the error is piped to the standard console output without crashing the host process.

6. Distribution Checklist

Before shipping your application, ensure the following files are located in the same directory as your .exe:

1. MD_BridgeLib.dll
2. python311.dll
3. python311._pth
4. The Lib/ directory